서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 전 보고서

STM32 기초 실습 및 FPGA 개별 실습

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 10월 11일
대상	예제1: 1초 간격 LED · 예제2: 버튼을 누르면 LED 변경 · STM32 과제1 · STM32 과제2 · 4비트 곱셈기 실습 · BCD 이진수 변환 실습
검증환경	VS Code 1.137.0 / STM32 CMake/ VaporView / AMD Vivado 2026.1 XSim

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab4

제출 태그: lab4-pre-v2

커밋 해시: a62ae01966e0fa68303296bdc15c0baa30489fa0

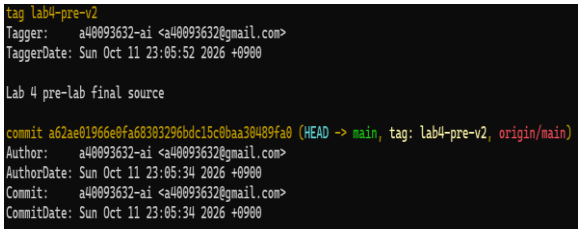


그림 1: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

본 실험의 목적은 STM32 보드에서 GPIO 입출력과 시간 기반 polling 방법을 이해하고, 버튼 입력을 이용하여 LED의 상태와 점멸 주기를 제어하는 것이다. 먼저 제공 예제를 통해 HAL_GetTick(), HAL_GPIO_ReadPin(), HAL_GPIO_WritePin(), HAL_GPIO_TogglePin()의 사용 방법을 확인한다. 이후 제공 예제를 수정하여 버튼을 누를 때마다 LED 점멸 주기가 1초, 2초, 4초, 8초 순서로 변경되도록 구현하고, 동일한 기능을 외부 LED가 연결되는 PB0 핀으로 확장한다.

또한 FPGA 개별 실습에서는 Verilog HDL을 이용하여 두 개의 4-bit 입력을 곱하는 조합논리 회로와 BCD 두 자리를 binary 값으로 변환하는 조합논리 회로를 확인한다. 각 회로는 testbench를 이용한 전수검사를 수행하고, PASS 메시지와 파형을 통해 출력이 예상 값과 일치하는지 검증한다.

실험 범위

	실습 내용	사전 검증 방법
예제1	PA5 LED를 1초 간격으로 반전	CMake 빌드
예제2	PC13 버튼을 누를 때 PA5 LED 상태 반전	CMake 빌드
과제1	버튼 입력마다 점멸 주기 1-2-4-8초 변경	CMake 빌드
과제2	과제 1의 출력을 PB0 외부 LED로 변경	CMake 빌드
실습1	4-bit x 4-bit 곱셈기	Verilog 시뮬레이션
실습2	두 자리 BCD를 2진수로 변환	Verilog 시뮬레이션

2. 관련 이론

HAL_GetTick()을 이용한 비차단 시간 제어

HAL_GetTick() 은 시스템이 시작된 후 경과한 시간을 ms 단위로 반환한다. 현재 시간과 마지막 동작 시간을 비교하면 HAL_Delay()로 전체 프로그램의 실행을 중단하지 않고 일정한 시간 간격을 만들 수 있다.

$$\Delta t = t_{now} - t_{last}$$

$$\Delta t \geq T_{interval}$$

위 조건이 만족되면 LED 상태를 변경하고 마지막 동작 시간을 현재 시간으로 갱신한다. 이 방식에서는 메인 루프가 계속 반복되므로 LED의 시간을 측정하는 중에도 버튼 입력을 확인할 수 있다.

BCD와 이진수 변환

BCD는 하나의 10진 숫자를 4비트 이진수로 표현하는 방식이다. 4비트로 표현 가능한 값은 0부터 15까지이지만 BCD에서는 0부터 9까지만 유효하다.[1] 십의 자리 ten과 일의 자리 one이 모두 유효하면 변환 결과는 다음과 같다.

$$bin = 10 \times ten + one$$

$$valid = (ten \leq 9) \wedge (one \leq 9)$$

두 자리 BCD의 최대 정상값은 99이며 $99 < 2^7$이므로 출력은 7비트 bin[6:0]으로 표현할 수 있다. 전체 256개 입력 조합 중 정상 입력은 10 x 10 = 100개이고, 비정상 입력은 256-100=156개이다.

3. 개발 및 검증 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
편집기	VS Code 1.137.0
파형 확인	VaporView
FPGA Part	Xc7s75fgga484-1
FPGA 도구	AMD Vivado 2026.1
보드	Combo II-DLD S75
STM32 빌드	CMake 기반 STM32
STM32 대상	NUCLEO-G474RE

주요 핀

핀	용도
PA5	보드 내장 LED 출력
PB0	외부 LED 출력
PC13	보드 사용자 버튼 입력
GND	외부 회로 공통 접지
PA13, PA14	프로그램 다운로드 및 디버깅
PA2, PA3	PC 연결 UART
PC10, PC11	FPGA 연결 UART

4. STM32 제공 예제1

실습 내용

제공 예제 1은 HAL_GetTick()으로 현재 시간을 확인하고, 이전 LED 변경 시점으로부터 1,000 ms가 경과했을 때 led_state를 SET과 RESET 사이에서 반전하는 프로그램이다. 계산된 상태는 HAL_GPIO_WritePin()을 이용하여 PA5 내장 LED에 출력한다.

동작 흐름

1. 현재 tick 값을 읽는다.

2. 마지막 LED 반전 시점과 현재 시간의 차를 계산한다.

3. 1,000ms 이상 경과했는지 검사한다.

4. 조건이 만족되면 led_state를 반전하고 HAL_GPIO_Write Pin()으로 PA5에 출력한다.

5. 마지막 반전 시간을 현재 시간으로 갱신한다.

예상 결과

경과 시간	PA5 LED
초기 상태	OFF
약 1초	ON
약 2초	OFF
약 3초	ON

사전 검증 결과

ece2_474_routine.c가 정상적으로 컴파일되었고 최종 실행 파일인 ece2_474.elf가 생성되었다. 빌드 결과에는 Build finished with exit code 0이 출력되었으므로 문법 오류와 링크 오류가 없음을 확인하였다.

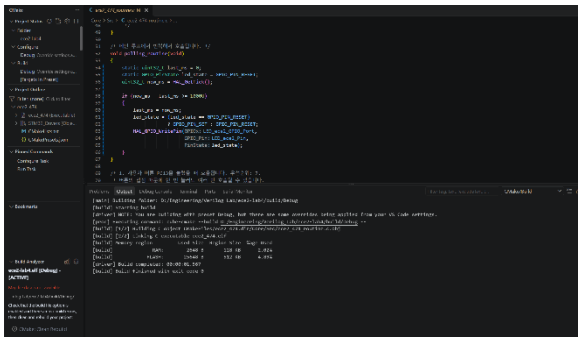


그림 2: STM32 제공 예제1 코드 및 CMake 빌드 결과

링크: 제공 예제 1 전체 코드

5. STM32 제공 예제2

실습 내용

제공 예제 2는 PC13 사용자 버튼의 상승 에지를 검출하여 PA5 내장 LED를 한 번 반전하는 프로그램이다. 버튼을 계속 누르고 있는 경우에도 LED가 반복해서 반전되지 않도록 현재 입력과 이전 입력을 비교한다. 또한 마지막 버튼 입력으로부터 200 ms 이상 경과한 경우에만 새로운 입력으로 인정하여 debounce를 구현한다.

동작 흐름

1. PC13 버튼 상태를 읽는다.

2. 현재 상태가 SET이고 이전 상태가 RESET인지 검사한다.

3. 마지막 버튼 인식 후 200ms가 지났는지 검사한다.

4. 조건이 모두 만족되면 PA5 LED를 한 번 반전한다.

5. 현재 버튼 상태를 이전 상태로 저장한다.

예상 결과

사용자 동작	예상 결과
버튼을 한 번 누름	LED 상태 한 번 반전
버튼을 계속 누름	추가 반전 없음
버튼을 놓고 다시 누름	LED 상태 다시 반전
짧은 bounce 발생	200ms 디바운스로 중복 입력 방지

사전 검증 결과

버튼의 현재 상태와 이전 상태를 이용한 상승 에지 검출 코드 및 200ms debounce 코드가 정상적으로 컴파일 되었다. ece2_474.elf가 정상 생성되었으며 빌드 종료 코드는 0이었다.

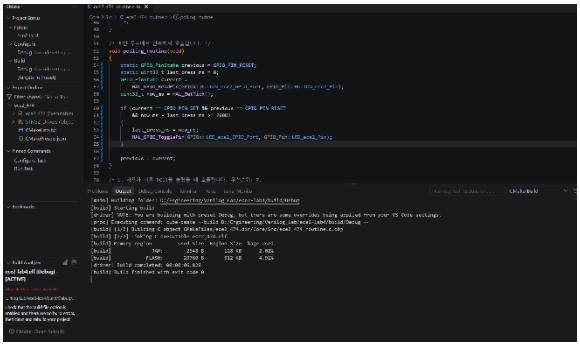


그림 3: STM32 제공 예제2 코드 및 CMake 빌드 결과

링크: 제공 예제 2 전체 코드

6. STM32 과제 1

설계 목표

과제 1에서는 버튼을 누를 때마다 PA5 내장 LED의 점멸 주기가 1초, 2초, 4초, 8초 순서로 변경되도록 한다. 8초 상태에서 버튼을 다시 누르면 1초로 돌아

오도록 순환 구조를 사용한다.

핵심 설계

LED 점멸을 위한 시간 변수와 버튼 debounce를 위한 시간 변수를 별도로 관리하였다. 버튼의 상승 에지가 검출되면 interval_ms를 두 배로 증가시키며, 값이 8,000 ms 이상이면 다시 1,000 ms로 초기화한다. 주기가 변경되는 순간 last_led_ms를 현재 시간으로 갱신하여 이전 주기의 남은 시간이 새로운 주기에 영향을 주지 않도록 하였다.

```
if (current == GPIO_PIN_SET && previous == GPIO_PIN_RESET
    && now_ms - last_press_ms >= 200U)
{
    last_press_ms = now_ms;

    if (interval_ms >= 8000U)
    {
        interval_ms = 1000U;
    }
    else
    {
        interval_ms *= 2U;
    }

    last_led_ms = now_ms;
}
```

그 림 4 : S T M 3 2 과 제 1 핵 심 코 드

예상 결과

버튼 입력 횟수	점멸 간격
초기	1초
1회	2초
2회	4초
3회	8초
4회	1초
5회	2초

사전 검증 결과

시간 비교, 상승 에지 검출, debounce 및 주기 변경 로직을 포함한 프로그램이 정상적으로 빌드 되었다. 최종 ELF 파일이 생성되었으며 빌드 종료 코드는 0이었다.

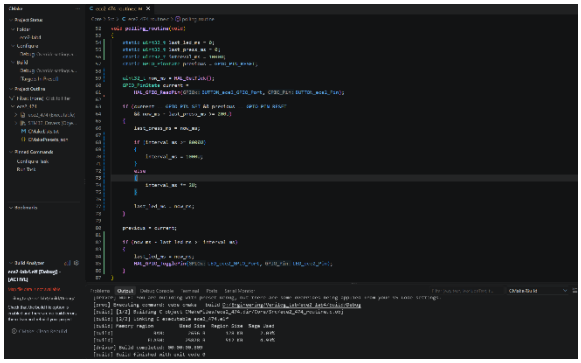


그림 5: STM32 과제 1 코드 및 CMake 빌드 결과

링크: [STM32 과제 1 전체 코드]

7. STM32 과제 2

설계 목표

과제 2에서는 과제 1의 시간 제어 및 버튼 처리 기능을 유지하면서 출력 핀을 PA5 내장 LED에서 PB0 외부 LED로 변경한다. 이를 통해 동일한 프로그램 구조를 다른 GPIO 출력에 적용하는 방법을 확인한다.

핵심 설계

과제 1의 전체 논리 구조는 그대로 유지하고, LED를 반전하는 GPIO 포트와 핀을 LED_ece2에서 GPIO_OUT1_ece2로 변경하였다. PB0는 프로젝트의 IOC 설정에서 범용 출력으로 설정되어 있으며 초기 출력은 LOW이다.

```
HAL_GPIO_TogglePin(GPIO_OUT1_ece2_GPIO_Port, GPIO_OUT1_ece2_Pin);
```

그 림 6 : S T M 3 2 과 제 2 핵 심 코 드

사전 검증 결과

PB0 출력 핀을 사용하도록 수정한 프로그램이 정상적으로 컴파일되고 링크되었다. ece2_474.elf 생성과 빌드 종료 코드 0을 확인하였다.

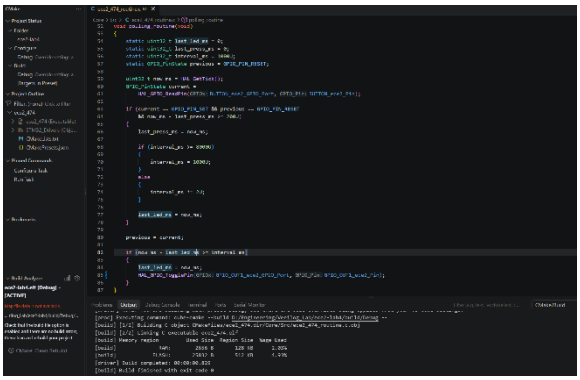


그림 7: STM32 과제 2 코드 및 CMake 빌드 결과

링크: [STM32 과제 2 전체코드]

링크: [STM32 전체 빌드 증빙 자료]

8. FPGA 실습 1 · 4비트 곱셈기

설계 목표

두 개의 4-bit unsigned 입력 a, b를 곱하여 8-bit 출력 m을 생성하는 조합논리 회로를 확인한다. 입력 또는 출력 저장을 위한 clock이나 register는 사용하지 않는다.

```
assign product = a * b;
```

그림 8: 4비트 곱셈기 핵심 코드

테스트벤치 검증 방법

Testbench에서는 a와 b를 각각 0부터 15까지 변화시켜 총 256개의 입력 조합을 검사한다. DUT 출력 m과 testbench에서 계산한 a*b가 일치하지 않으면 오류를 출력하고, 모든 입력이 일치하면 PASS 메시지를 출력하도록 구성하였다.

예상 결과

a	b	예상 m
0	15	0
3	5	15
9	9	81
15	15	225

검증 결과

시뮬레이션 결과 PASS multiplier: 256 input pairs가 출력되어 전체 256개 입력 조합에서 예상 결과와 DUT 출력이 일치함을 확인하였다. 파형에서는 a=3인 상태에서 b가 0부터 F까지 증가할 때 출력 m이 0, 3, 6, 9, ..., 2D의 16진수 값으로 증가하였다. 마지막 값 2D는 10진수 45이므로 3 x 15 = 45와 일치한다.

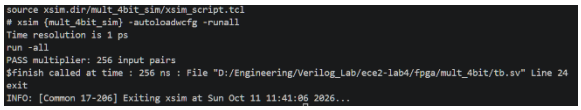


그림 9: 4비트 곱셈기 Verilog PASS 결과

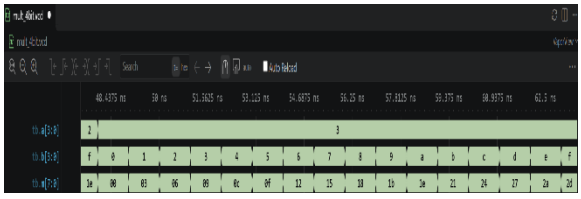


그림 10: 4비트 곱셈기 시뮬레이션 파형

링크: 4비트 곱셈기 전체 소스

링크: 4비트 곱셈기 증빙 자료

9. FPGA 실습 2 · BCD Converter

설계 목표

두 개의 4-bit 입력 ten과 one을 각각 십의 자리와 일의 자리로 해석하여 7-bit bin[6:0] 값으로 변환한다. 두 입력이 모두 0부터 9 사이인 경우에만 valid를 1로 출력한다. 비정상 BCD 입력에서도 조합식에 의해 bin에 어떤 값이 나타날 수 있지만 valid=0이므로 해당 값은 유효한 변환 결과로 사용하지 않는다.

```
assign binary = ten * 7'd10 + one;
assign valid = (ten <= 9) && (one <= 9);
```

그림 11: BCD Converter 핵심 코드

테스트벤치 검증 방법

Testbench에서는 ten과 one을 각각 0부터 15까지 변화시켜 전체 256개 입력을 검사한다. 100개의 유효한 BCD 입력에서는 상위 모듈 출력 bin이 ten×10+one과 일치하고 valid=1인지 확인하고, 156개의 잘못된 BCD 입력에서는 valid=0인지 확인한다.

예상 결과

Ten	One	예상 bin	Valid
0	0	0	1
3	7	37	1
9	9	99	1
A	0	무효 입력	0
5	F	무효 입력	0

검증 결과

시뮬레이션 결과 PASS BCD: 100 valid and 156 invalid inputs가 출력되었다. 따라서 100개의 정상 BCD 입력과 156개의 비정상 입력이 모두 예상대로 판정되었음을 확인하였다. 파형에서도 일의 자리 값이 0부터 9일 때 valid=1로 유지되고, A부터 F까지에서는 valid=0으로 변경되었다.

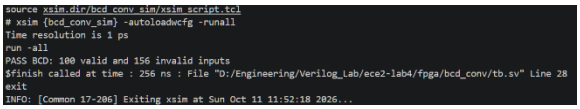


그림 12: BCD 컨버터 Verilog PASS 결과



그림 13: BCD 컨버터 시뮬레이션 파형

링크: BCD Converter 전체 소스

링크: BCD Converter 증빙 자료

10. 사전 검증 결과 및 문제 해결

사전 검증 결과 종합

실습	예상 결과	사전 검증 결과
제공 예제1	PA5 LED가 1초마다 반전	CMake 빌드 성공
제공 예제2	버튼 1회당 PA5 한 번 반전	CMake 빌드 성공
STM32 과제 1	간격이 1-2-4-8초로 순환	CMake 빌드 성공
STM32 과제 2	PB0 외부 LED가 같은 간격으로 동작	CMake 빌드 성공
4비트 곱셈기	256개 입력 조합 모두 일치	PASS multiplier: 256 input pairs
BCD 변환기	100개 valid, 156개 invalid	PASS BCD: 100 valid and 156 invalid inputs

문제 해결 과정

STM32 프로젝트를 일반 코드 폴더로 열었을 때 VS Code 왼쪽에 CMake 메뉴가 나타나지 않았다. 프로젝트에서 제공하는 워크스페이스 파일로 다시 열어 CMake와 STM32 프로젝트 구성이 인식되도록 하였다.

FPGA 시뮬레이션 과정에서는 VCD 파일이 완전히 생성되지 않았거나 생성 경로가 일치하지 않을 때 파형 프로그램에서 invalid 파일로 표시되었다. wave.tcl에서 VCD 파일을 열고, 신호를 기록하고, 시뮬레이션을 끝까지 실행한 뒤 VCD 파일을 닫도록 설정하여 최종 파형을 확인하였다.

11. 실험 당일 검증 계획

STM32 실험에서는 각 코드를 다시 빌드한 후 F5를 이용하여 보드에 다운로드한다. 제공 예제 1에서는 PA5 LED가 1초 간격으로 변화하는지 확인하며, 제공 예제 2에서는 버튼을 길게 누르고 있어도 LED가 한 번만 반전되는지 확인한다. 과제 1에서는 버튼 입력마다 점멸 간격이 1초, 2초, 4초, 8초 순서로 변경되는지 확인한다. 과제 2에서는 PB0에 저항과 외부 LED를 연결하고 동일한 간격 변경이 나타나는지 확인한다.

FPGA 실습에서는 교재의 핀 배치표를 이용하여 각 회로의 XDC를 작성한다. 4비트 곱셈기는 mult_4bit, BCD 변환기는 bcd_conv를 Design Top으로 설정한다. 이후 Synthesis, Implementation 및 Generate Bitstream을 순서대로 수행하고 실제 보드 출력과 시뮬레이션 결과를 비교한다.

실습	실험 당일 확인할 항목
예제1	PA5 LED가 1초마다 상태 변경
예제2	버튼을 길게 눌러도 한 번만 반전
과제1	버튼 입력마다 1-2-4-8-1초 순환
과제2	PB0 외부 LED에서 같은 간격 변경
4비트 곱셈기	대표 입력의 곱 확인
BCD 변환기	정상, 비정상 BCD의 bin과 valid 확인

12. 결론

본 실험을 준비하면서 STM32 HAL 함수를 이용한 GPIO 입출력 방법과 HAL_GetTick()을 이용한 비차단 시간 제어 방법을 확인하였다. 또한 현재 버튼 상태와 이전 버튼 상태를 비교하는 상승 에지 검출 및 200 ms debounce를 적용하여 버튼을 한 번 누를 때 한 번의 동작만 수행하도록 구성하였다. 이를 바탕으로 LED 점멸 간격을 1초, 2초, 4초, 8초 순서로 변경하는 프로그램과 PB0 외부 LED 출력 프로그램을 작성하였으며, 두 프로그램 모두 오류 없이 빌드되었다.

FPGA 실습에서는 4비트 곱셈기와 BCD 이진수 변환기의 계층적 조합논리 구조를 확인하였다. 4비트 곱셈기는 256개의 전체 입력 조합을 통과하였으며, BCD 변환기는 100개의 정상 입력과 156개의 비정상 입력을 모두 정확하게 판정하였다. 현재 STM32와 FPGA 실제 보드를 보유하고 있지 않아 하드웨어 동작은 확인하지 못하였다. 실험 당일 STM32 프로그램 다운로드와 외부 LED 연결을 수행하고, FPGA에서는 XDC 작성과 비트스트림 생성을 진행하여 사전 검증 결과와 실제 보드 동작을 비교할 예정이다.

참고 문헌

[1] P. Penfield Jr., *Information and Entropy*, Ch. 2, "Codes," Massachusetts Institute of Technology OpenCourseWare, 2008.